Wing Span

Overview

Our team is creating a digital version of Wingspan, a strategic board game that focuses on collecting bird cards, managing resources, and earning points through habitats and bonus objectives. The project combines Java programming, object-oriented design, and GUI development to simulate a playable version of the game for four players.

Project Goals

- *Build a fully functional, bug-free version of Wingspan using Java.*
- *Create a complete object-oriented structure (Game, Player, Board, BirdCard, etc.).*
- *Integrate a GUI interface (MenuPanel and GamePanel) that connects directly with the backend logic.*
- *Implement a turn-based system and a score calculation algorithm that mirrors the official rulebook.*
- *Develop automated testing data to verify each function and ensure consistent performance.*

Team Communication

Communication: Messages

We will call twice a week to discuss how the project is doing.

Weekly Development Plan

Week 1 – Foundation and Structure
Set up project folders, classes, enums, and UML relationships. Each member focuses on building constructors and placeholder methods to define the program's structure.

Week 2 – Integration of Core Elements
Connect the BirdCard and BonusCard enums with the Player and Game classes. Implement deck logic, card distribution, and drawing/discarding mechanics.

Week 3 – Game Flow and Turn System
Develop methods for startGame(), nextTurn(), nextRound(), and endGame(). Begin testing the turn order and round progression while setting up the score calculation system.

Continued....

Week 4 – GUI Creation
Design the MenuPanel and GamePanel, code mouse interactions, and connect the visual layer with backend logic. Ensure that all interface actions (clicking, transitions, and displays) respond in real time.

Week 5 – Full Integration and Refinement
Merge the logic and GUI completely. Finalize the scoring system, apply BonusCard effects, and polish all player actions to achieve a smooth gameplay loop.

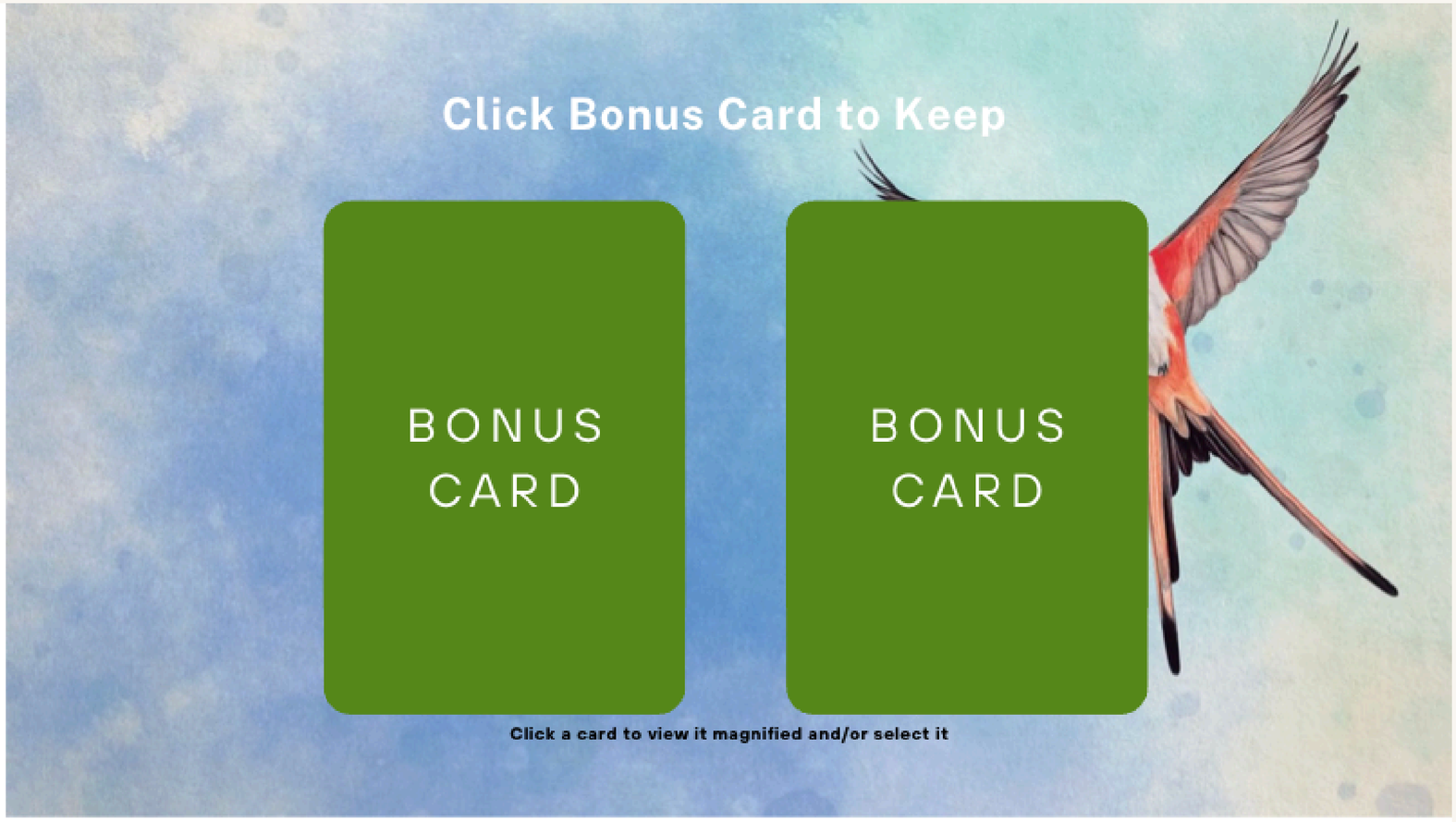
Week 6 – Testing and Optimization
Run logic and graphics tests to eliminate runtime errors. Test edge cases such as full habitats, max eggs, and empty decks. Optimize performance and fix GUI glitches.

Week 7 – Final Review and Playtest
The entire team playtests the game to confirm stable performance, correct scoring, and visual consistency. Final polish ensures the game runs smoothly from start to finish.

Start Screen



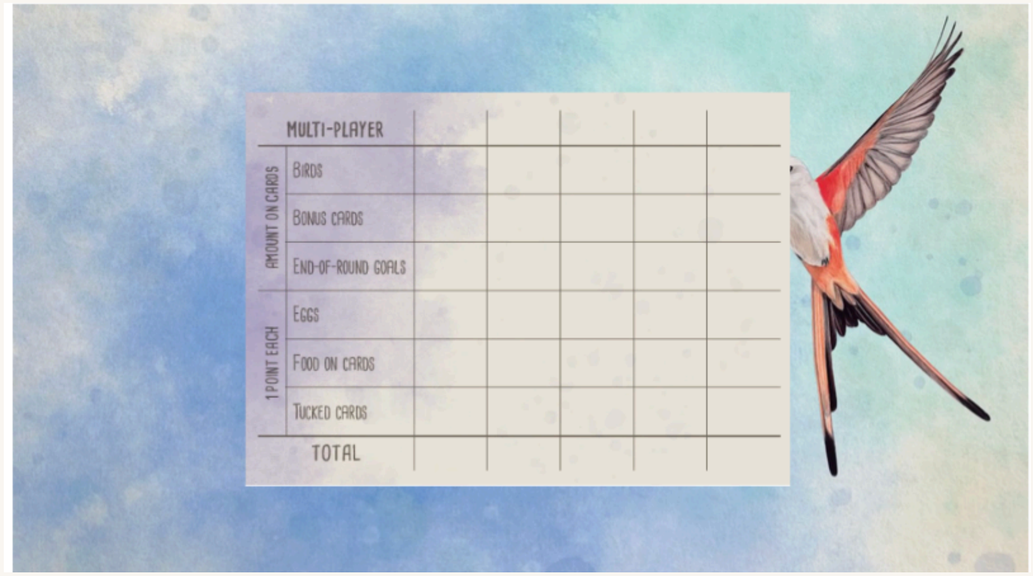
Bonus Card Selection



Game Screen

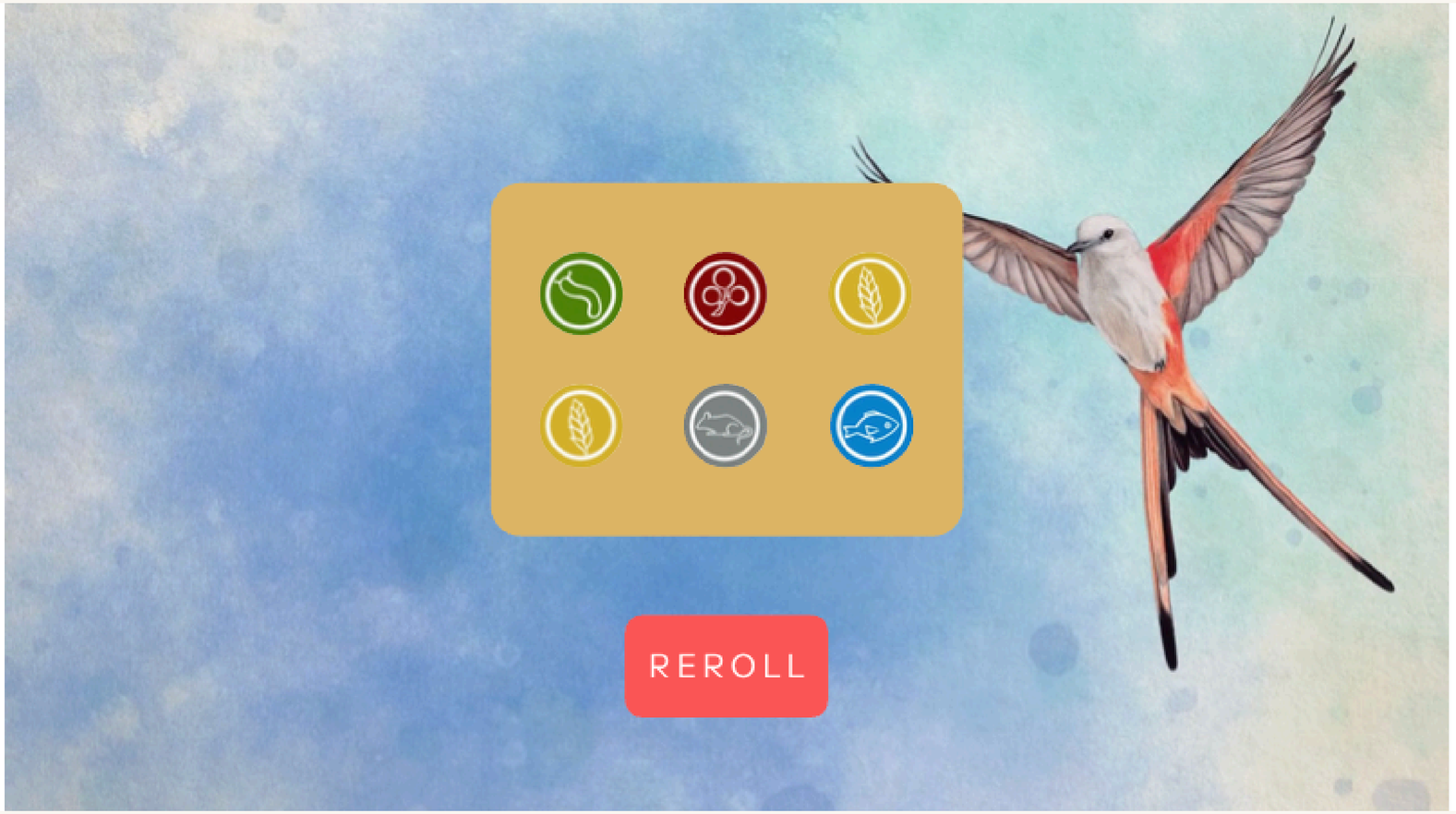

PLAY A BIRD
GAIN FOOD
FROM BIRDFEEDER
SPENT
GAME END: 5/2 PTS FOR MAJORITY
THEN ACTIVATE ANY BROWN POWERS IN THIS ROW
LAY EGGS
ON BIRD(S)
SPENT
GAME END: 5/2 PTS FOR MAJORITY
THEN ACTIVATE ANY BROWN POWERS IN THIS ROW
DRAW BIRD CARDS
SPENT
GAME END: 5/2 PTS FOR MAJORITY
THEN ACTIVATE ANY BROWN POWERS IN THIS ROW
Round scoring
CUBES ON "PLAY A BIRD"
Active Player: Player 1
Viewing: Player 2
Round: 1/3
Action Cubes
Food tokens
x2
x3
x1
x4
x2
Click to view previous or next player
Player Hand
CARD
CARD
CARD
CARD
CARD
CARD
BONUS CARD
BONUS CARD
BONUS CARD
Click a card to view it magnified and/or select it

Score/Results





Food Selection



Front End UML

Front end (what players see and click)

Purpose: Render the game; capture user input; reflect back-end state instantly.

Responsibilities:

Draw the board, cards, eggs, food, and icons with clear colors per habitat.
Handle clicks/hover (play bird, lay eggs, gain food, draw cards, end turn).

Show immediate feedback: highlights for legal moves, error messages (e.g., “Wrong habitat”, “Insufficient food”), and live HUD updates.

Keep things readable and responsive at classroom/projector sizes.

WingSpan+MenuPanel

Wingspan
Hosts the main window and swaps screens.
Shows MenuPanel first; switches to GamePanel on “Play”.
Calls into the back end (e.g., startGame, nextTurn) and then refreshes the view.

MenuPanel
Title Screen: “Click to Play”.
Setup Screen
Start Selection:
Pick card and food takens and makes sure it adds up to five.
“Click to magnify” preview for any card.
Bonus Card Selection: click to preview; choose one to keep.

GamePanel (main play screen)

- *Habitat grid (3×6): top row = Forest (Gain Food), middle = Grassland (Lay Eggs), bottom = Wetland (Draw Cards). Each cell shows eggs/food/card art when present.*
- *HUD / Round Scoring (top-right):*
- *Active Player, Viewing Player, Round (e.g., 1/4), Action Cubes left.*
- *Food token strip (invertebrate, seed, fish, rodent, fruit).*
- *Arrows to view previous/next player board.*
- *Player Hand (right column):*
- *Blue tiles = Bird cards; green tiles = Bonus cards.*
- *Visual highlights for legal/illegal placements.*
- *Messages for errors (e.g., “Wrong habitat”, “Insufficient food”, “Row full”).*
- *Immediate updates after every action (cubes, eggs, food, cards).*
- *BirdFeeder view (if shown):*
- *Dice faces reflect current feeder roll; reroll prompt when all same.*

Back End UML

Game

Purpose: Owns players, decks, turn/round flow, and end-game.

Spot

Purpose: Holds one bird plus its eggs, cached food, and tucked cards.

- *assignCard()–Set this spot's occupying bird(if currently empty).*
- *addEgg()–Adds egg count up to that bird's eggMax.*
- *removeEgg()–Decreases egg count(not below zero).*
- *addFood(int[] food)–Add cached food to this spot's food array.*
- *occupied()–True if bird is present.*

BirdFeeder

Purpose: Models feeder dice and reroll rule by the Forest action.

- *roll()–Randomize five dice values(1-6).*
- *allSame()–Boolean, True if all dice show the same face(enables the reroll condition)*

BirdCard(enum)

Purpose: Defines all birds and their immutable properties.

- *BirdCard(String name, int maxEgg, int habitats, double wingspan, int[] foodCost, int actionId, boolean onPlay) — Create an enum constant with card stats (name, egg cap, habitat bitmask, wingspan, cost, effect id, trigger timing).*

Testing and Validation

Our testing process includes both logic-based and graphics-based verification:

- *Ensuring turn order, score calculation, and deck operations work correctly.*
- *Checking that GUI elements (HUD, buttons, icons) update instantly after actions.*
- *Running test cases for all major functions such as playing cards, gaining food, and laying eggs.*
- *Recording results in the Testing Data section for documentation and debugging.*

Research Results



60% of students sleep less than 6 hours on school nights

Students who sleep 7–8 hours tend to have higher test scores

75% of sleepy students said they find it hard to focus in morning classes

Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam tempor sapien eu mauris viverra bibendum. Nam sodales, sapien et aliquet luctus, libero mauris mollis odio, ac vulputate orci tellus vitae enim.

Suggestions

Try to sleep at least 7 hours, especially on school nights

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam tempor sapien eu mauris viverra bibendum. Nam sodales, sapien et aliquet luctus, libero mauris mollis odio, ac vulputate orci tellus vitae enim.

Avoid screens 1 hour before bedtime

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam tempor sapien eu mauris viverra bibendum. Nam sodales, sapien et aliquet luctus, libero mauris mollis odio, ac vulputate orci tellus vitae enim.

Thank You